

Installation et utilisation de ST Model Zoo

Adam Guglielmino

1 Installer la suite STM32Cube

Les logiciels ST principaux sont CubeIDE, CubeMX et CubeProgrammer. Les 2 derniers peuvent être utilisés directement au sein de l'IDE. **Ces trois logiciels ne sont pas nécessaires pour utiliser Model Zoo services.** Ils permettent seulement de créer ou de modifier les applications et de déboguer la carte.

Ils sont téléchargeables à partir d'ici :

<https://www.st.com/en/development-tools/stm32cubeide.html#tools-software>

En téléchargeant la version pour linux générique, on se retrouve avec trois fichiers suivants :

- stm32cubmx-lin-v6-15-0.zip
- stm32cubeprg-lin-v2_20-0.zip
- st-stm32cubeide_1.19-0_(...).zip

Les placer dans un dossier (/STM32Cube par exemple).

1.1 STM32CubeProgrammer

Ce logiciel sert à configurer les registres et à flasher les fichiers dans la mémoire des cartes de développement.

Installation :

- extraire l'archive et exécuter SetupSTM32CubeProgrammer(...).linux

```
mkdir CubeProgrammerSetup
unzip stm32cubeprg(...).zip -d CubeProgrammerSetup
cd CubeProgrammerSetup
./SetupSTM32CubeProgrammer(...).linux
```

- Utiliser le programme pour :
 - Accepter la licence et les CGU
 - Étape 5 : Choisir le dossier d'installation (/STM32Cube/STM32CubeProgrammer par exemple)
 - Étape 6 : Sélectionner les trois modules : Core File, STM32CubeProgrammer, et enfin STM32TrustedPackageCreator

Le programme pourra être exécuté, par exemple, depuis :

~/STM32Cube/STM32CubeProgrammer/bin/STM32CubeProgrammer

On retrouvera également dans ce dossier la version Command Line Interface (CLI) de CubeProgrammer : STM32_Programmer_CLI et l'outil STM32_SingingToom_CLI qui pourra être utilisé pour signer les applications avant de les flasher.

1.2 STM32CubeIDE

Ce logiciel est l'application de développement principale, elle comprend les fonctionnalités de STM32CubeMX et STM32CubeProgrammer.

Installation :

```
unzip st-stm32cubeide(...).sh.zip
chmod +x st-stm32cubeide(...).sh
sudo ./st-stm32cubeide(...).sh
```

Ensuite, il faut accepter la licence et préciser le dossier d'installation (/home/<user>/STM32Cube/stm32cubeide-1.19.0 par exemple). Puis, accepter l'installation de STLinkServer et sa licence.

CubeIDE pourra ensuite être exécuté depuis /home/<user>/STM32Cube/stm32cubeide-1.19.0/stm32cubeide

1.3 STM32CubeMX

Ce logiciel permet de configurer un projet en choisissant les pins et les bibliothèques puis de générer le code nécessaire à leur utilisation. Il peut être utilisé au sein de STM32CubeIDE.

Installation :

- Extraire l'archive
- lancer SetupSTM32CubeMX-6.15.0
- Choisir le dossier d'installation (/STM32Cube par exemple)
- extraire l'archive et exécuter SetupSTM32CubeMX(...)

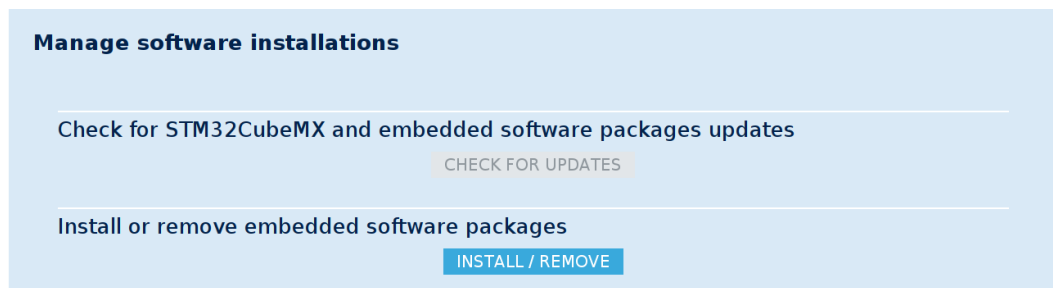
```
mkdir CubeMXSetup
unzip stm32cubemx(...).zip -d CubeMXSetup
cd CubeMXSetup
./SetupSTM32CubeMX(...)
```

- Utiliser le programme pour :
 - Accepter la licence et les CGU
 - Choisir le dossier d'installation (/STM32Cube/STM32CubeMX par exemple)
- CubeMX pourra être exécuté depuis /STM32Cube/STM32CubeMX/STM32CubeMX

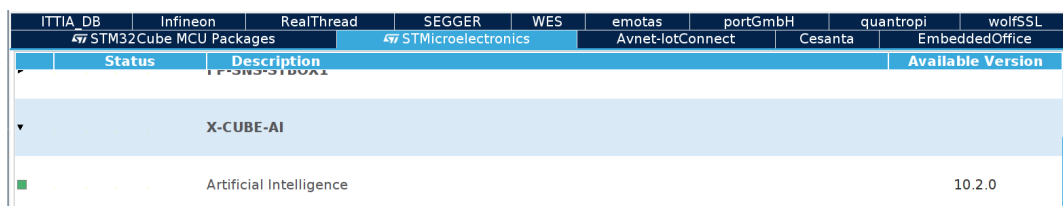
1.3.1 Installation de X-Cube-AI

CubeMX peut générer des applications basées sur ST EdgeAI Core avec le pack X-Cube-AI. Une application de validation est disponible ainsi qu'une application template.

- Dans le menu de CubeMX, cliquer sur Install/Remove dans la section Manage software installations



- Une fenêtre s'ouvre, **rafraichir**
- Dans l'onglet STMicroelectronics, dérouler le menu X-Cube-AI et sélectionner le package Artificial Intelligence (version 10.2.0).



- Se connecter à son compte myST pour finir l'installation

2 ST Edge AI et Model Zoo

ST fournit des logiciels et une banque de modèles pour utiliser des modèles d'IA sur le STM32N6. Toute la toolchain de Model Zoo services est supportée pour certains formats de modèles (.h5, .hdf5, .keras, .onnx, .tflite). ST Edge AI est nécessaire pour pouvoir compiler les modèles. Le logiciel peut être téléchargé en entier avec CubeMX ou bien seulement sa composante CLI. Seul cette dernière est utilisée par Model Zoo Services.

Lien vers la documentation : <https://stedgeai-dc.st.com/assets/embedded-docs/index.html>

2.1 ST Edge AI Developer Cloud

Le cloud de ST Edge AI permet de faire du benchmarking plus complet, notamment de connaître le temps d'inférence du modèle. Il se trouve ici :

<https://stedgeai-dc.st.com/home>.

2.2 Installer ST Edge AI

Le chemin d'installation de ST Edge AI devra être spécifiée dans le fichier de configuration pour pouvoir utiliser Model Zoo services.

Ne pas utiliser d'espace dans le chemin d'accès

- Télécharger STEdgeAI Core : <https://www.st.com/en/development-tools/stedgeai-core.html>
- Extraire l'archive et lancer le fichier installeur *stedgeai-linux-onlineinstaller*. L'installation nécessite un compte myST.
- Choisir le dossier d'installation
- Sélectionner les composants suivants : ST EdgeAI Core (version 2.2.0), STM32 MCU module et ST Neural-ART module.
- Télécharger le module STEdgeAI-NPU comme demandé et spécifier son chemin d'installation.
- L'installation est prête à être lancée

Le fichier se trouve sous le chemin spécifié lors de l'installation.

2.3 Installer Model Zoo

Model Zoo est accompagné de services permettant d'entraîner des modèles sur des datasets locaux, de quantifier, d'évaluer et de déployer plusieurs types de modèles d'IA. Il existe 2 dépôts github :

- Les scripts et services : <https://github.com/STMicroelectronics/stm32ai-modelzoo-services>.
- Les modèles de références : <https://github.com/STMicroelectronics/stm32ai-modelzoo>

1. Télécharger les dépôts github en utilisant git lfs :

2. Note : Une installation classique avec le SSH fonctionne très bien normalement

- Dans un terminal, se déplacer dans le dossier d'installation, par exemple : `mkdir /ModelZoo` et `cd /ModelZoo`
- Installer git et git-lfs avec les commandes `sudo apt-get update`, `sudo apt-get install git`, `sudo apt-get install git-lfs`
- Exécuter : `git lfs install`
- Cloner les dépôts : `git clone https://github.com/STMicroelectronics/stm32ai-modelzoo.git` et `git clone https://github.com/STMicroelectronics/stm32ai-modelzoo-services.git`

3. Installer l'environnement python :

- Vérifier que python est installé et que sa version est $> 3.10.X$:

```
python --version
```

- Si python n'est pas installé, l'installer :

```
sudo apt install python3 python-is-python3
```

- Se déplacer dans la racine de Model Zoo services : `stm32ai-modelzoo-services`. Puis créer et activer l'environnement python.

```
cd stm32ai-modelzoo-services
python -m venv st_zoo
source st_zoo/bin/activate
```

- Enfin, installer les prérequis :

```
pip install -r requirements.txt
cd speech_enhancement
pip install -r torch_requirements.txt
pip install onnx==1.16.1 protobuf==3.20.2
```

4. Description des modèles de speech enhancement :

Les modèles se trouvent dans le dossier : `stm32ai_modelzoo/speech_enhancement/` (...). 5 fichiers se trouvent sous ce dossier : 3 modèles préentraînés onnx (.onnx), un fichier `state_dict` pour entraîner et évaluer des modèles torch, et un template de fichier de configuration (.yaml).

Les trois modèles sont :

- Le modèle float32 à taille dynamique (à quantifier pour déployer).
`stft_tcnn_medium_sigmoid_257xsl_float.onnx`
- Le modèle int8 à taille dynamique (déjà quantifié, pour l'évaluation et le benchmarking. Non déployable).
`stft_tcnn_medium_sigmoid_257xsl_qdq_int8.onnx`
- Le modèle int8 à taille statique (à déployer).
`stft_tcnn_medium_sigmoid_257x40_qdq_int8.onnx`

3 Speech enhancement

Les modèles pré-entraînés se trouvent dans Model Zoo. Ils se trouvent selon ce chemin : `stm32ai-modelzoo/speech_enhancement/stft_tcnn/ST_pretrainedmodel_public_dataset/valentini`
Pour faciliter la configuration, ils peuvent être copiés dans un nouveau dossier : `stm32ai-modelzoo-services/speech_enhancement/pretrained_models/valentini`

3.1 Configuration de Model Zoo services

Toute la configuration de l'application speech enhancement se fait grâce au fichier suivant : `stm32ai-modelzoo-services/speech_enhancement/user_config.yaml`

Le fichier est prérempli, il faut cependant y apporter des modifications :

- Pour préciser le chemin vers le modèle à évaluer, quantifier ou déployer, il faut remplir le paramètre dans `model` : **`onnx_path`** .
- Remplir le chemin vers *stedgeai* et *stm32cubeide* comme ceci :

```
tools:
  stedgeai:
    version: 10.2.0 # At least 10.0.0
    optimization: balanced
    on_cloud: False
    path_to_stedgeai: /home/<user>/STM32Cube/Repository/Packs/STMicroelectronics/X-CUBE-AI/10.2.0/Utilities/linux/stedgeai
    path_to_cubeIDE: /home/<user>/STM32Cube/stm32cubeide_1.19.0/stm32cubeide
```

- Changer le chemin vers le projet C, le chemin par défaut est faux :
c_project_path : ../application_code/audio/STM32N6/
- **Changer le nom de la configuration de compilation**, les noms proposés par défaut ne sont pas les noms des configurations utilisées réellement par le script. Les noms réels sont ci-dessous :

```
build_conf : "BM" # N6 Audio Bare Metal, this is the default configuration
# build_conf : "FREERTOS" # N6 Audio Thread X
# build_conf : "BM_LP" # N6 Audio Bare Metal Low Power
# build_conf : "FREERTOS_LP" # N6 Audio Thread X Low Power
```

Ce fichier est également utilisé pour choisir quelle opération exécuter. Les choix possibles sont parmi : [training, evaluation, quantization, benchmarking, deployment]

Des chaînes d'opération sont aussi possibles : [chain_tqe, chain_eqe, chain_tqeb, chain_eqeb, chain_qd]

3.2 Entraînement

Le seul type de modèles utilisable avec Model Zoo services est le type STFT-TCNN, il s'agit d'une modification du TCNN dans le spectre fréquentiel.

3.2.1 Dataset

Le dataset utilisé pour l'entraînement des modèles fournis par Model Zoo est le dataset Valentini. Il est téléchargeable ici : <https://datashare.ed.ac.uk/handle/10283/2791>.

Pour faciliter la configuration, il peut être placé dans un nouveau dossier :

~/datasets/valentini

Les archives à télécharger et à extraire dans le nouveau dossier **valentini** sont :

- clean_testset_wave.zip
- clean_trainset_56spk_wave.zip
- noisy_testset_wav.zip
- noisy_trainset_56spk_wav.zip

3.2.2 Configuration

Il faut tous d'abord remplir le fichier **user_config.yaml**.

- Changer le mode d'opération : **operation_mode : training**
- Passer sur le CPU si besoin : **device : cpu**
- Pour éviter les crashes pendant l'entraînement : **num_data_loader_workers : 0**
- Spécifier le chemin relatif vers la racine du dataset :

```
dataset:
  name: valentini # Or "custom"
  root_folder: datasets/valentini # Root folder of dataset
  n_speakers: 56 # For Valentini, 28 or 56 speaker dataset. Does nothing if name is "custom"
  file_extension: '.wav' # Extension of audio files. Valentini dataset uses .wav
```

3.3 Évaluation

Cette opération sert à obtenir différentes métriques sur le modèle. Il est évalué sur le dataset spécifié dans le fichier de configuration. L'étape de configuration du dataset décrite dans la section 3.2 est donc nécessaire.

Dans le fichier `user_config.yaml` :

- Changer le mode d'opération : **operation_mode : evaluation**
- Dans la section évaluation, spécifier le paramètre `fixed_sequence_length` si le modèle évalué est de taille statique.

3.4 Quantifier

Cette opération permet de quantifier un modèle, cette étape est nécessaire pour le déploiement.

- Changer le mode d'opération : **operation_mode : quantization**
- Spécifier le chemin relatif du modèle onnx à quantifier, avec le paramètre `onnx_path`, dans la section `model`. Par exemple :

```
pretrained_models/valentini/stft_tcn_medium_sigmoid_257xsl_float.onnx
```

Le script précise où est sauvegardé le modèle quantifié. La configuration de ce chemin se fait avec le paramètre `hydra : run : dir :` et `general : saved_models_dir :`. Par défaut dans :

```
./src/experiment_outputs/$now :%Y_%m_%d_%H_%M_%S/saved_models
```

3.5 Déploiement

Pour déployer le modèle sur la carte :

- La carte se connecte au PC par le port USB-C ST-LINK_V3EC
- Passer la carte en mode *dev boot* : Basculer le switch BOOT1 vers la droite puis débrancher/rebrancher.
- Changer le mode d'opération dans le fichier de configuration `user_config.yaml` : **operation_mode : deployment**
- Préciser le chemin du modèle quantifié statique avec le paramètre `onnx_path`, dans la section `model`.

Une fois le programme flashé, pour le lancer, il faut basculer les 2 switchs vers la gauche et déconnecter puis reconnecter le câble USB.

3.6 Exécution

Une fois la configuration faite, le script peut être exécuté.

- Activer l'environnement python :

```
cd stm32ai-modelzoo-services
source st_zoo/bin/activate
```

- Exécuter le script :

```
cd speech_enhancement
python stm32ai_main.py
```

4 Liens utiles

Les informations sur comment utiliser la toolchain de Model Zoo Services sont ici :

`stm32ai-modelzoo-services/tree/main/speech_enhancement/docs`

Le dataset utilisé pour l'entraînement de ces modèles est le dataset Valentini. Il est téléchargeable ici :

<https://datashare.ed.ac.uk/handle/10283/2791>

Les informations sur le réseau sont ici :

https://github.com/STMicroelectronics/stm32ai-modelzoo/tree/main/speech_enhancement/stft_tcn